



Module 21

Introduction to a Modern Verification Flow

cā dence®

This training module introduces you to a modern verification methodology.

Objectives

In this module, you will:

- Identify the need for and nature of a Metric-Driven Verification (MDV) flow.

Topics include:

- Examining the verification problem
- Automatic stimulus generation with randomization
- Coverage to measure what has and has not been tested
- Assertions to check for correct and incorrect behavior
- Defining a Verification Plan



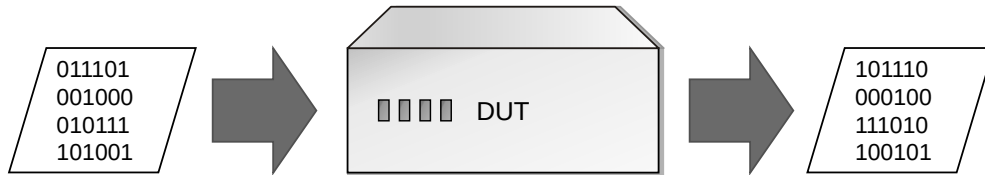
In this training module, you explain the need for, and nature of, the MDV methodology.

What Is the Verification Problem?

The **verification problem** is how to scale the verification effort to keep pace with demand.

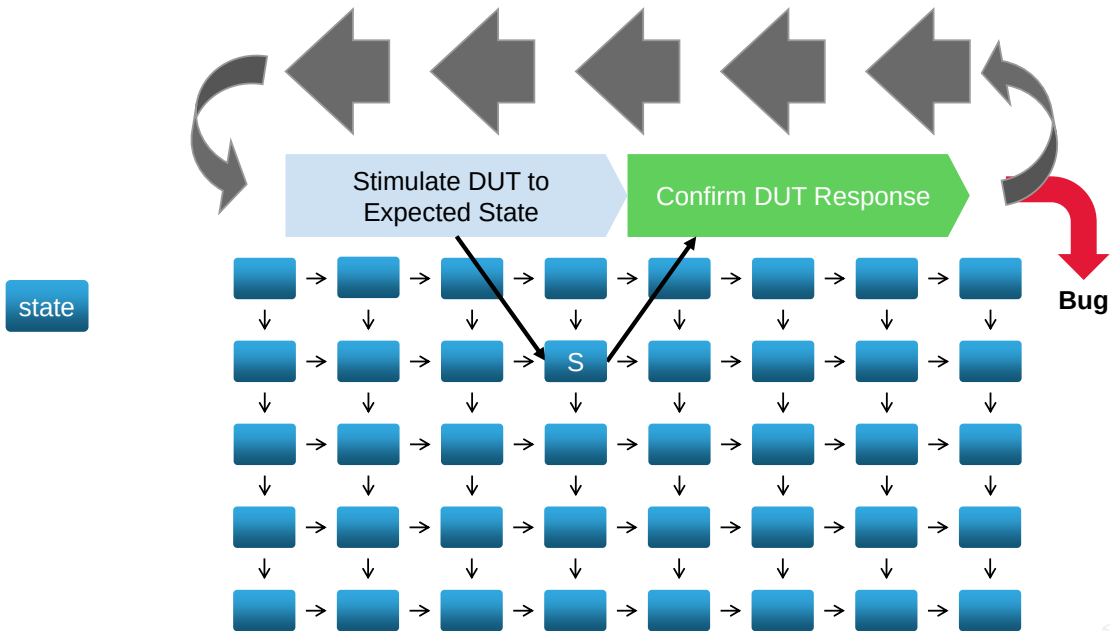
The fundamental verification activity is to stimulate the DUT and check the DUT response.

- The problem is how to scale this activity to keep pace with continually increasing DUT size and complexity.



The verification effort is at its most abstract level to control and observe, that is, apply stimulus and confirm the correct response.

Typical Verification via Simulation



4

© Cadence Design Systems, Inc. All rights reserved.



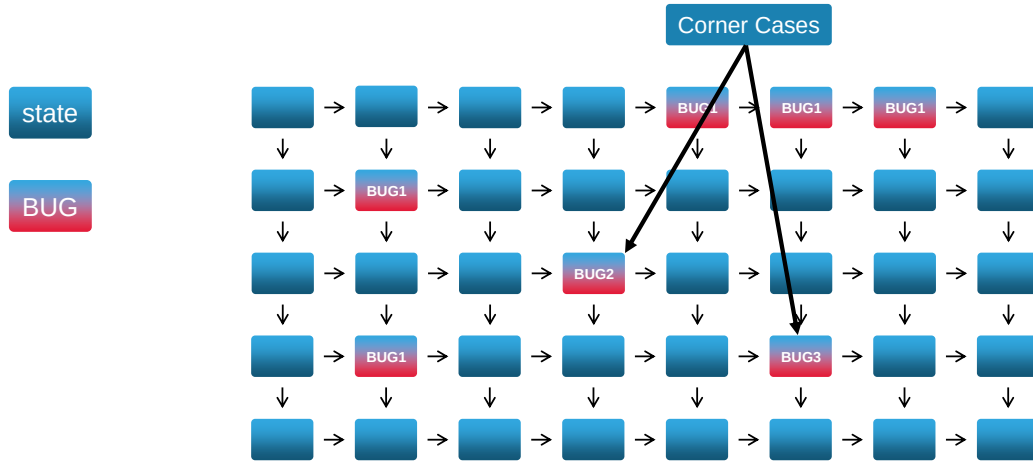
Typical verification via simulation stimulates the DUT to what we hope is a specific state and then confirms the DUT state and outputs. A design “bug” can prevent the DUT from achieving the expected state or producing something other than the expected output.

Where Are the Bugs?

Most “bugs” are quickly found and fixed.

The elusive bugs are revealed only by “interesting” states.

These are typically the “corner case” states.



5

© Cadence Design Systems, Inc. All rights reserved.



Where do the “bugs” hide? Most bugs are quickly found. Elusive bugs are revealed only by unusual states. These are typically the corner case states less frequently visited.

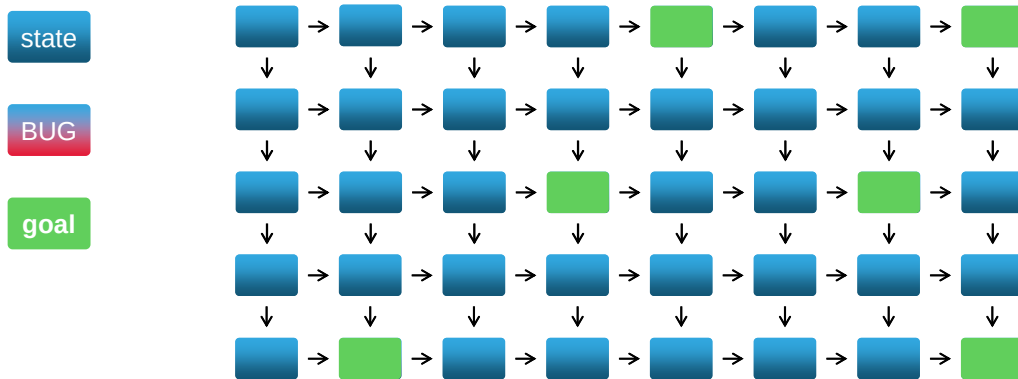
Illustrated here are one easily detected bug and two corner-case bugs.

Verification Goal States

The verification goal is to reveal the bugs.

To reveal bugs, the verification team establishes “goal” states to visit.

They do this based on analyzing the specification, identifying key features, and team experience.



6

© Cadence Design Systems, Inc. All rights reserved.

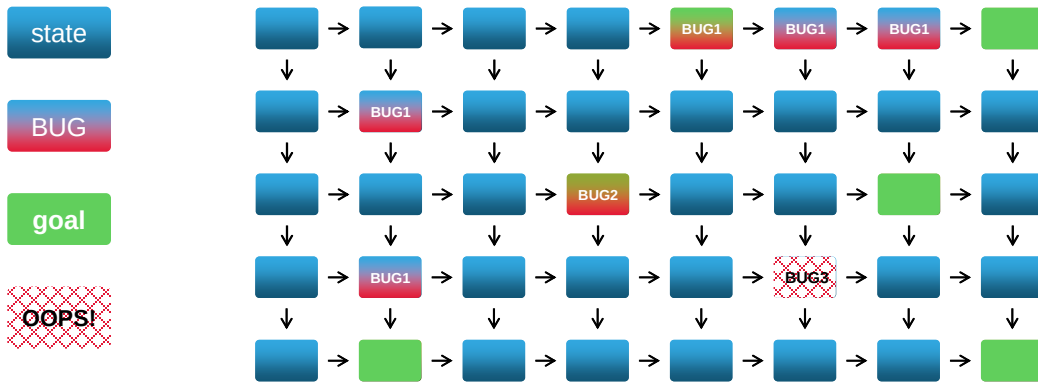


The verification goal is to reveal the bugs. To reveal bugs, the verification team establishes “goal” states to visit. It determines these states based on analyzing the specification, identifying key features, and applying team experience.

Here, the team has selected some goal states to visit.

Hit and Missed Bugs

- Verification goals may not reveal all bugs.
- Some bugs may be revealed only by states outside of the goals.
- Here, the verification plan missed a corner-case bug.
- We must not limit verification to only the *known* goals!



7

© Cadence Design Systems, Inc. All rights reserved.



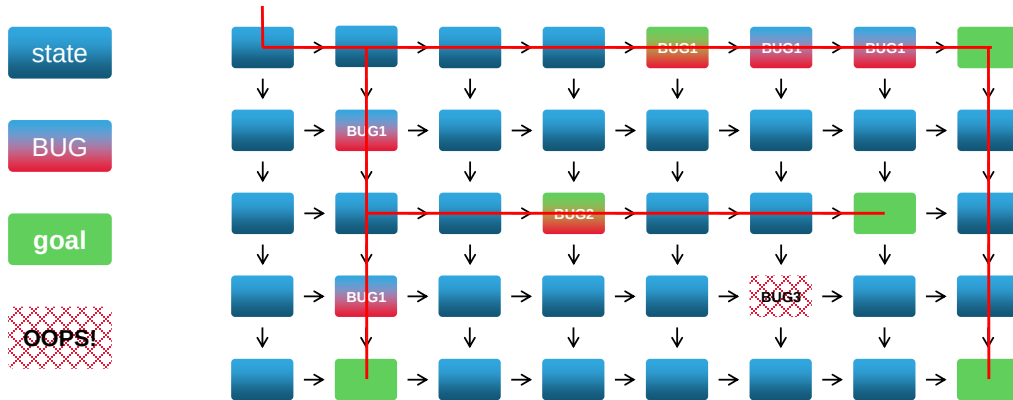
The verification goals hit some bugs and miss some bugs. Some bugs may be revealed only by states outside of the verification goals. Here, the verification plan missed a corner-case bug.

We must not limit verification to only the known goals!

Legacy Directed Test “Gate Banging”

The verification engineer wrote a directed test for each item in the verification plan.

- **No Automation** – Manual effort required to determine and follow the paths.
 - **No Automation** – Manual effort required to verify that goals were reached.
 - **No Automation** – Non-goal scenarios unlikely to be exercised.
- Repeat for every design change!



8

© Cadence Design Systems, Inc. All rights reserved.

cadence

Traditionally, the verification engineer wrote a directed test for each item in the verification plan. The test would apply a stimulus and confirm the response. None of the efforts was automated.

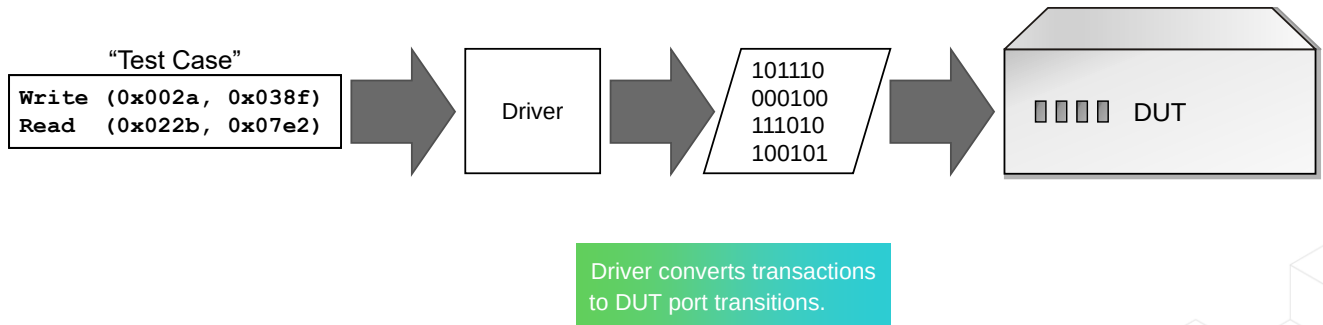
- The manual effort was required to determine and follow the paths;
- The manual effort was required to verify that goals were reached; and
- Non-goal scenarios were unlikely to be exercised.

Here, the test implemented the verification plan. It revealed the easily detected bug and a corner-case bug but missed the unplanned-for corner case bug.

The Case for Transaction-Based Verification

Moving stimulus generation to a higher level of abstraction improved productivity.

- The verification engineer wrote tests calling SystemVerilog tasks (VHDL procedures, C functions).
- The manual effort was required to determine and follow the paths (now via parameter selection).
- The test stimulus intent was still not well documented.
- The expected response was still typically built into the test.



9 © Cadence Design Systems, Inc. All rights reserved.



Moving stimulus generation to a higher level of abstraction improved test writing productivity.

- The verification engineer wrote tests calling SystemVerilog tasks (VHDL procedures or C functions).
- The manual effort was still required to determine and follow the paths, now via parameter selection;
- The test stimulus intent was still not well documented; and
- The expected response was still typically built into the test.

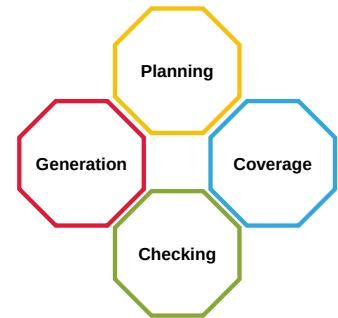
Here the test case calls a write routine, passing to it the address and data to write, and calls a read routine, passing to it the address and expected data.

What Is the Verification Problem Solution?

The verification problem **solution** is **Metric-Driven Verification** or **MDV**!

MDV is a powerful layer of methodology that sits above the Verification Testbench environment, which provides guidelines and tools for using metrics and automation to maximize the benefits of the verification testbench.

- It is a data-driven, decision-based flow that improves the Predictability, Productivity and Quality of the verification effort.
- MDV is a **closed-loop process** with the following stages:
 1. **PLAN** an Executable Verification Plan.
 2. **CONSTRUCT** a testbench and other verification infrastructure using Universal Verification Methodology (UVM).
 3. **EXECUTE** the regression on various engines such as Simulation, Formal Verification, Emulation, etc.
 4. **MEASURE AND ANALYZE** the results to see what else is left to be done to finish.



The Metric-Driven Verification (MDV) methodology has four key components:

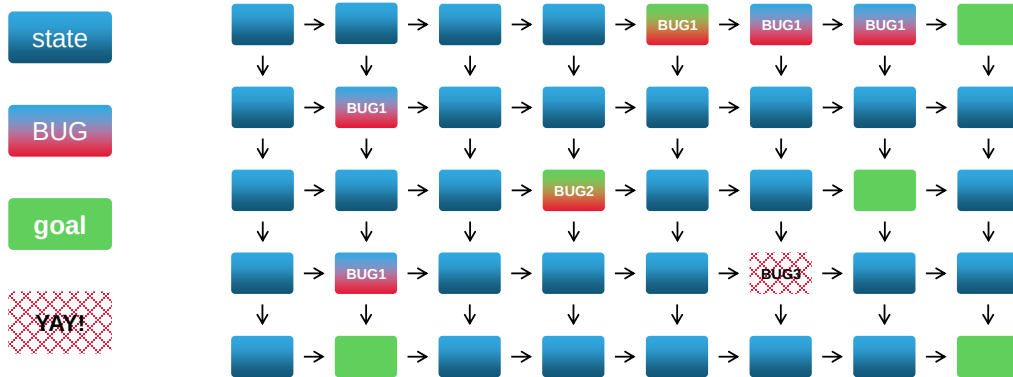
- Planning your verification goals and measuring your progress toward them.
- Generation of random stimulus constrained to meaningful values and ranges of values;
- Checking to confirm that the design reacted appropriately to the stimulus;
- Coverage to confirm that the stimulus is generated, and exercised design features as planned;

Generation with Constrained Randomization

Random stimulus (biased toward goal areas) partially automates test generation.

- **Automation** – Less manual effort required to determine and follow the paths.
- **Automation** – Non-goal scenarios more likely to be exercised.
- **No Automation** – Manual effort *still* required to verify that goals are reached.
- **No Automation** – Manual effort *now* required to check for expected results.

Repeat for every design change!



11

© Cadence Design Systems, Inc. All rights reserved.

cadence

To partially automate test generation, verification engineers began to incorporate random stimulus, biased toward the goal areas.

This automated or semi-automated some of the verification process.

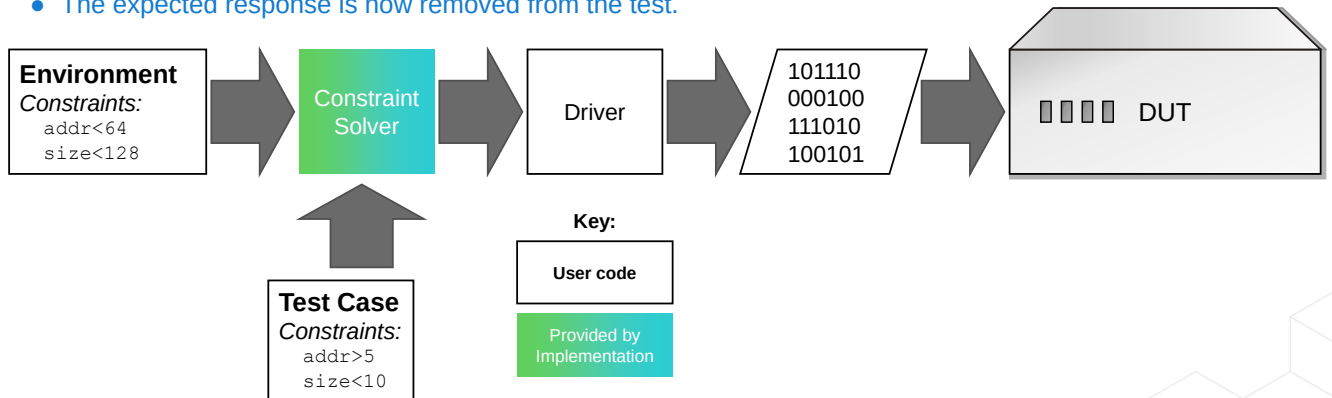
- The less manual effort was required to determine and follow the paths; and
- Non-goal scenarios were more likely to be exercised; but
- The manual effort was *still* required to verify that the goals were reached; and
- The manual effort was *now* required to cull the ineffective stimulus.

Here, the verification plan included biased random stimulus. Its implementation revealed the easily detected bug and both corner-case bugs, including the unplanned-for corner case bug.

Constrained Random Stimulus

Your test case is constraint sets.

- The *Verification Environment* specifies the *allowable* value ranges.
- Each *Test Case* specifies the *desired* value ranges.
- The test stimulus intent is now better documented.
- The expected response is now removed from the test.



12 © Cadence Design Systems, Inc. All rights reserved.



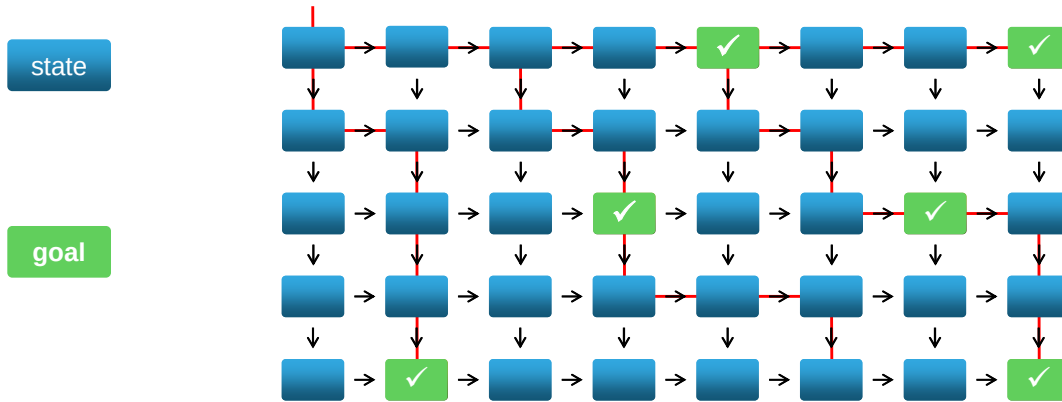
With constrained random stimulus, your test case is constraint sets.

- The *Verification Environment* specifies the allowable value ranges;
- Each *Test Case* specifies the desired value ranges within the allowable ranges;
- Constraint comments and names now better document the test stimulus intent; and
- The expected response is now removed from the test case.

Coverage Measures: What Is Tested?

Coverage partially automates verification that goals are reached.

- **Automation** – Coverage verifies what is and what is not tested.
- **Automation** – Coverage enables culling the ineffective stimulus.
- **No Automation** – Manual effort to add coverage to your verification environment.



13 © Cadence Design Systems, Inc. All rights reserved.



This page does not contain notes.

Coverage Makes Automatic Stimulus Generation Effective

Without coverage measurement...

- Lacks objective test goal achievement feedback.
- Many constraint sets provide little confidence that specific scenarios have been reached.
- No feedback on the effectiveness of constraint sets, leading to duplication of effort.

With coverage measurement...

- Has objective test goal achievement feedback.
- Fewer constraint sets reach most scenarios.
- Greater efficiency of verification through the removal of redundant or ineffective constraint sets.



Constrained random stimulus is most effective when paired with coverage.

Without coverage data, you have little or no idea what tests, if any, exercised what, if any, scenario.

With coverage data, you know which tests are useful and which design scenarios remain not exercised.

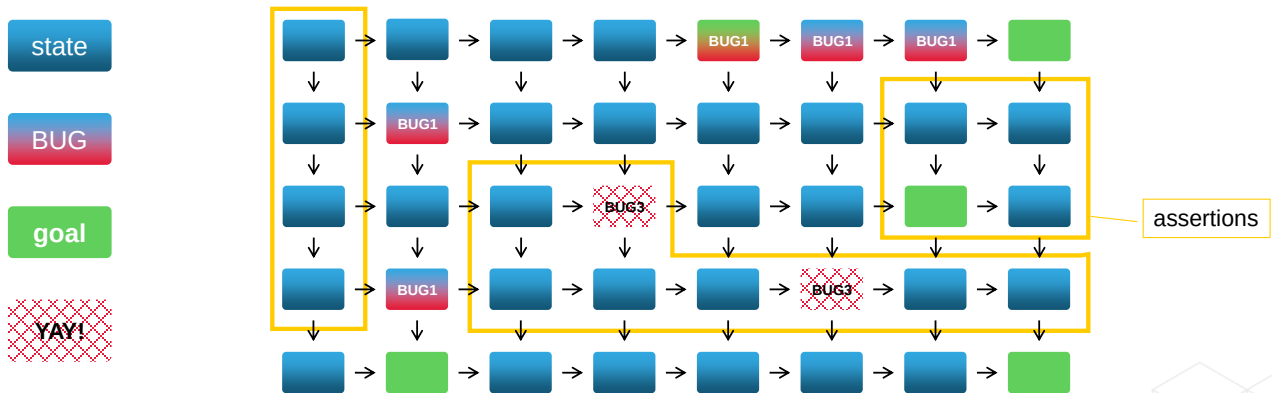
Assertions Check Behavior

Expected results for constrained random stimulus may not be known.

Self-checking can be embedded into DUT with assertions.

- **Automation** – Assertions check for good and bad behavior.
- **No Automation** – Manual effort to add assertions to your verification environment.

Checking



15

© Cadence Design Systems, Inc. All rights reserved.

cadence

With random stimulus, it may not be possible to construct expected data sets against which DUT values can be checked. The solution is to build self-checking into the test environment and the DUT itself using assertions. Assertions are embedded checkers, written in SystemVerilog, which can check for good behavior and detect bad behavior during the course of simulation. As well as checking, assertion syntax can also be used for coverage measurement and in formal verification.

Metric-Driven Verification Benefits: Automation

Design complexity continually increases....

...Therefore, verification complexity continually increases.

Manual effort cannot continue to increase at the same rate.

Application of manual effort must instead be at increasingly abstract levels.

- Providing stimulus goals.
- Analyzing coverage data.
- Analyzing checker-reported errors.
- Providing more direction when goals are not being met.

Automation

16

© Cadence Design Systems, Inc. All rights reserved.



Verification complexity continually increases.

The manual effort cannot continue to increase at that rate.

The manual effort must instead be made more productive by being applied at increasingly abstract levels.

- Providing stimulus goals;
- Analyzing checker-reported errors; and
- Providing more direction when goals are not being met.

This “providing more direction” should ideally be as automated as practical.

Metric-Driven Verification Benefits: Completeness

Verification complexity makes it impossible to imagine all failure scenarios.

We need a way to exercise failure scenarios other than those imagined.

Constrained random stimulus tests a DUT more thoroughly than directed tests.

Random stimulus is most effective when paired with coverage.

Completeness

17

© Cadence Design Systems, Inc. All rights reserved.



Verification complexity makes it impossible to imagine all failure scenarios.

We need a way to exercise failure scenarios other than those imagined. Constrained random stimulus focuses on goal areas but also as an unplanned side-effect exercises scenarios that the test plan, and thus directed test, might have missed.

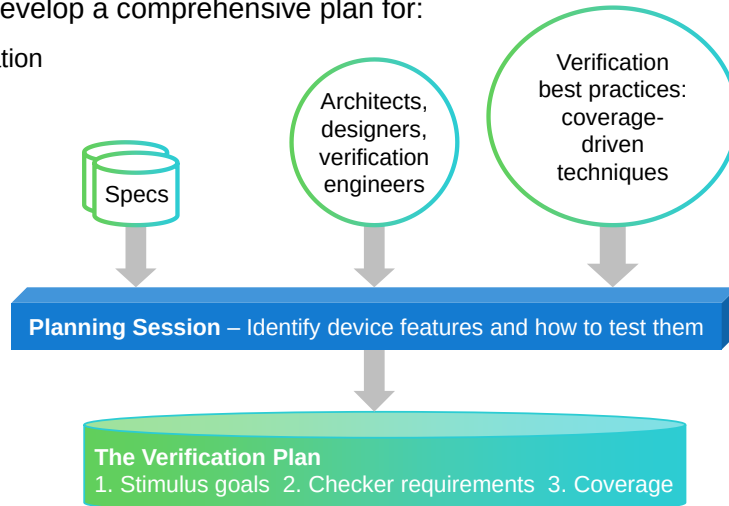
Constrained random stimulus is most effective when paired with coverage. Coverage confirms whether the goals are reached and enables ranking the stimulus sets and eliminating ineffective or redundant stimulus sets.

The Verification Plan

So, you want to use the MDV methodology but don't forget your *planning*.

Using a planning process, develop a comprehensive plan for:

- Automated stimulus generation
- Independent checking
- Coverage collection

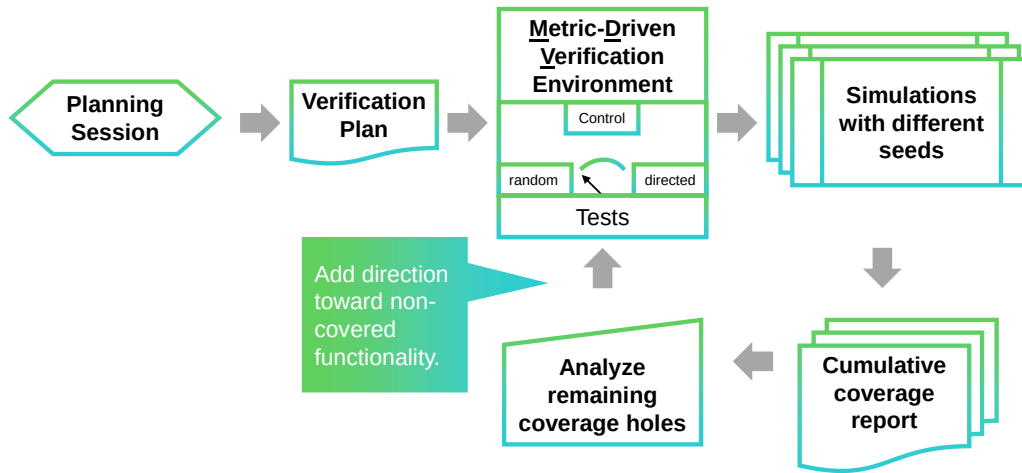


So you want to use the MDV methodology, but don't forget your *planning*.

Using a planning process that draws upon appropriate resources, develop a comprehensive plan for:

- Automated stimulus generation;
- Independent checking; and
- Coverage collection.

The Verification Process



19 © Cadence Design Systems, Inc. All rights reserved.



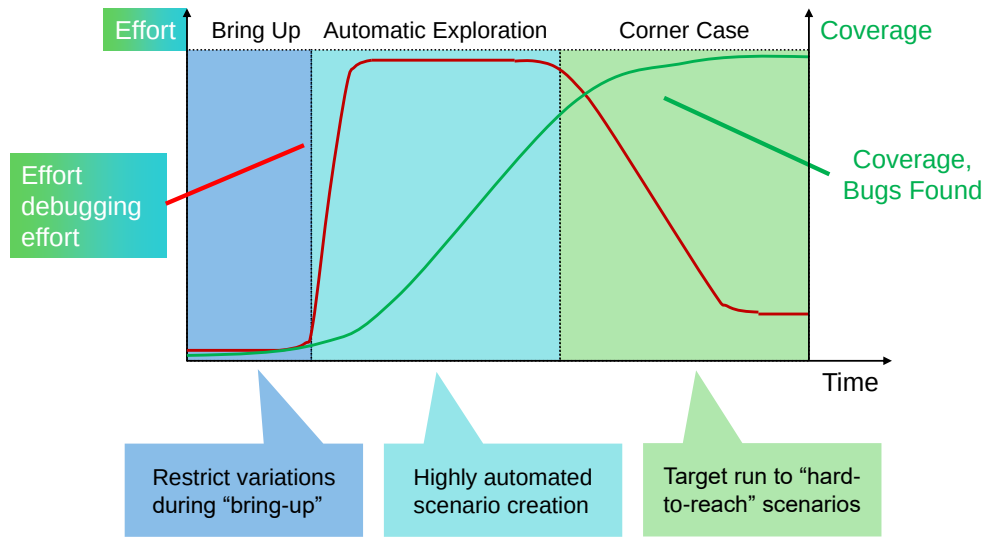
From the verification plan, you develop or reuse a verification environment and develop tests. Recall that tests are constraint sets. The tests can be as flexible, or not, as you want them to be.

You feed the verification environment and tests into multiple simulations with different seeds.

Each simulation generates coverage data, which you then analyze to determine where the coverage holes are, and also to cull the ineffective or redundant tests.

To address the coverage holes, you add tests with constraints modified to address those goal areas.

Verification Project Lifetime



20 © Cadence Design Systems, Inc. All rights reserved.



This sketch somewhat describes what you can expect to put into, and get out of, your test development efforts.

Every new verification project starts with a short interval in which you are “bringing up” the verification environment. At this point, your constraints have little variation and you are not really detecting and fixing too many bugs.

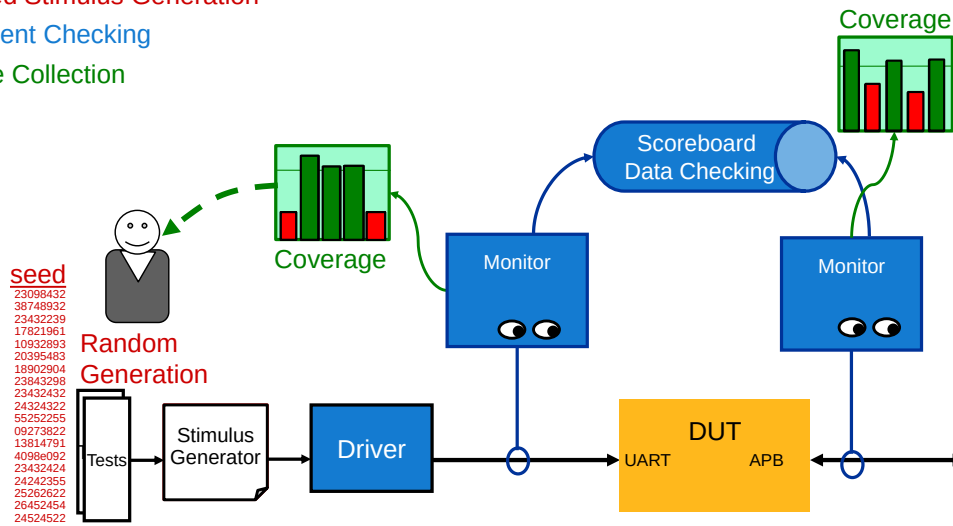
After “bringing up” the verification environment, a longer interval follows in which tests are automatically generated in accordance with your expanded constraint variations, while your team is spending much of its time discovering and fixing how neither the design nor their understanding of the specification really matches the specification.

Lastly, the project starts to wind down, the debug effort drops off, as some team members leave to bring up other projects, and the remaining team members spend sleepless nights in Seattle wringing out the last few bugs while total coverage asymptotically approaches 100%.

Metric-Driven Verification Review

Components of a MDV environment

- Automated Stimulus Generation
- Independent Checking
- Coverage Collection



21 © Cadence Design Systems, Inc. All rights reserved.

cadence

A Metric-Driven Verification methodology is composed of three main features:

1. Random stimulus to test the widest possible design functionality with the smallest stimulus code.
2. Assertions and independent checking to define behavior which should or should not happen, and so find bugs in the design.
3. Coverage so we can determine what has and has not been tested. Coverage feeds back to the test writer, who can then modify the stimulus to close coverage holes.

Class-Based Testbench

Traditional testbenches are written using a hierarchy of modules:

- Just like RTL code.

SystemVerilog adds classes:

- C/Java-like, Object-Oriented (OO) software-like programming constructs.
- Used for writing testbenches and simulation models.

Why do we use classes for testbench development?

- Easier code reuse.
- Configuration and customization.
- Powerful support for constrained randomization, coverage and checking.
- Pre-written class library of verification building blocks, simulation control, debugging features, etc.:
 - Universal Verification Methodology (UVM)

22 © Cadence Design Systems, Inc. All rights reserved.



The SystemVerilog language includes C/Java-like class constructs for Object-Oriented (OO) design. The class features of SystemVerilog are not synthesizable, and therefore not used in RTL design, but are used for writing testbenches and simulation models.

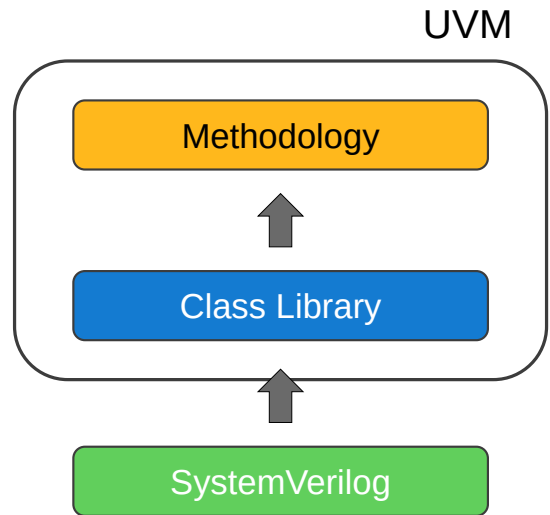
Using classes for testbench creation provides many significant advantages:

- Classes can be more easily packaged for internal and commercial reuse.
- Object-Oriented programming allows you to redefine almost any part of a testbench using inheritance and subclasses, while leaving the original code untouched. This allows easy, safe and convenient reconfiguration and customization of your testbench components, giving far better flexibility for reuse.
- Constrained randomization, coverage and checking features are far more powerful in a class environment rather than a traditional module-based testbench.
- But the most powerful advantage is UVM, a standard class library of pre-written verification building blocks, simulation control and debugging features (and much more). UVM is basically an Application Programming Interface (API) for verification, providing a set of SystemVerilog subprograms, wrapped in classes, with which a verification engineer can build a testbench environment.

What Is UVM?

A class library of verification building blocks written in standard IEEE1800 SystemVerilog. Supports features such as phasing, messaging, factory, etc.

- A proven verification methodology:
 - Defines how to use the class library.
 - Scalable from block-level to system-level verification.
- IEEE standard 1800.1:
 - Documented and maintained by IEEE.
 - Open source reference library from Accellera.
 - Apache license.
 - Does not lock users into a single vendor solution.



23 © Cadence Design Systems, Inc. All rights reserved.



UVM is a library of SystemVerilog classes that implement verification building blocks along with support features such as phasing, messaging, factory, etc. The library provides additional class automation, such as print, copy, comparison, cloning, etc., which are not available in SystemVerilog.

The library API is defined as an IEEE standard (1800.1), and Accellera (the industry standard body) provides a reference implementation of the library. By extending our testbench environment from the UVM library, we can exploit the built-in automation and support features.

However, UVM is also a methodology for using the building blocks and features of the library. By following the methodology, you can create scalable verification components which can be used from block-level to system-level verification.

By following the methodology and the architecture it dictates, we can create standardized, reusable verification blocks.

UVM has an Apache license, which allows commercial IP to be built and sold on top of UVM.

All this means that companies can create internal, external and commercial verification IP conforming to UVM, hence allowing the greatest possible reuse.

UVM Library Benefits

Library classes provide verification functionality and building blocks.

For example:

- Structural components
 - Verification blocks supporting a standard architecture
- Automation
 - Automatic implementation of print, copy, compare, etc. subprograms
- Simulation control
- Reporting

Benefits:

- Productivity
 - How much manual work can be saved?
- Predictability
 - Will I meet my deadlines on time?
- Quality
 - Will my testbench be free of bugs?



UVM is a library of SystemVerilog classes that implement verification building blocks along with support features such as phasing, messaging, etc.

Automation refers to the automatic generation of common class operations such as copy, print and compare, which are not available in SystemVerilog. By generating these for us, UVM saves us from having to write these ourselves. In general, UVM provides the building blocks for testbench generation, saving us considerable time.

Developing a testbench using UVM building blocks allows us to leverage tried and tested code and therefore minimizes bugs and errors.

UVM Methodology

Methodology defines how to use library classes.

Benefits

- Proven solution for verification success.
- Captures best-known practices with minimal improvisation.
- Provides consistency and uniformity in the way verification components are developed and used.
 - Internal and commercial reuse can dramatically speed up delivery and improve quality.
- Accommodates unanticipated requirements.
 - Supports easy reconfiguration, customization and modification.
- Ease of use.
 - How easily can a Test Writer develop tests for an existing verification environment?
- Supports key verification features required for Metric-Driven Verification (MDV).

25 © Cadence Design Systems, Inc. All rights reserved.



Although the first version of UVM was only established in 2009, it leverages the verification methodology developed by Cadence's Specman[®] tool, which was released in 1996. So UVM has a proven success record and leverages decades of experience from prior tools.

A good methodology, based on a standard verification component architecture, increases productivity and predictability, as it saves much time otherwise taken in planning and partitioning the testbench design.

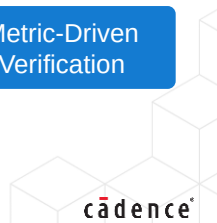
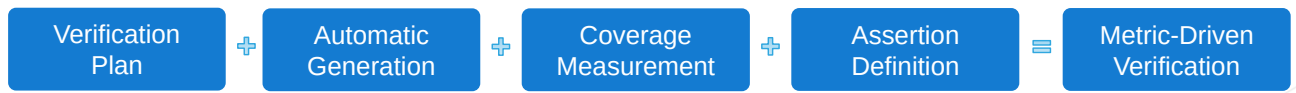
UVM recognizes that a reusable block or third-party verification IP may never completely match the requirements of a user and so provides powerful configuration and type-override capabilities to allow the user to customize and modify verification blocks.

A test writer is a verification engineer who develops tests, according to the verification plan, for an existing verification environment. UVM is designed to allow a test writer to have maximum control without requiring an in-depth knowledge of the environment. UVM also supports key verification features required for MDV.

Module Summary

You should now be able to explain the workings and benefits of the Metric-Driven Verification (MDV) methodology.

- Why you need an advanced verification methodology
- How the MDV methodology meets that need
- Implemented in your testbench using UVM



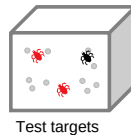
In this training module, you explained the need for and nature of the MDV methodology. You justified the effort that you will be putting into this training course. You examined why you need an advanced verification methodology, how the MDV methodology fills that need, and very briefly, to what you apply coverage metrics.

Functional Verification Methodologies Review

Directed Tests



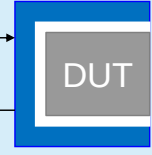
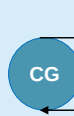
Results



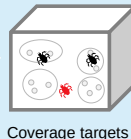
Test 1: ✓
Test 2: ✓
Test 3: ✗
Test 4: ✓

Least effective in finding the hidden bugs.

Coverage



Results



Coverage

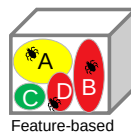
CG1 ✓
CG2 ✓
CG3 ✗
CG4 ✓

Adds quality and productivity, but difficult to estimate completion.

Metric Driven



Results



Feature-based verification targets
(Metrics for cov+checks)

Chip Features

Feature A
 Subset 1 50%
 Subset 2 20%
Feature B
Feature C
Feature D



Feature-based plan with extended metrics enables efficient and accurate project closure.

This training module initially discussed the legacy directed-test verification methodology. This methodology focuses on test generation. This methodology is now not viable due to the complexity of modern designs and the competitive pressures that prevent us from merely throwing more manpower at the verification problem.

The Metric-Driven Verification (MDV) methodology utilizes automated constrained-random generation checkers to confirm design operation and coverage to confirm test completeness. All are controlled via the planning process with verification project management tools.

This methodology focuses on a higher level of design features. With the project management tools, we know not just how complete the test is but also how much more time we need to achieve a specific level of completeness.